



Agentic Lifecycle Orchestration

Evidence-Gated Handoffs from Issue to Pull Request

Make every transition inspectable, authorized, recoverable, and owned

"How should independently governed workflows hand work off across an issue-to-PR lifecycle?"

Treat every workflow source as an uncompiled candidate until the target repository proves compatibility.

Compile first. Then inspect authority, evidence, recovery, and ownership.

Developer

Trace bounded implementation authority from plan to pull request.

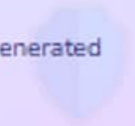
Team Lead

Keep approval and merge decisions with named humans.



Platform Engineer

Compile candidates and inspect generated workflow locks.



4 candidates

Workflow sources remain explicitly uncompiled before adoption.

1 exact approval

Only the named maintainer approval advances implementation.

0 hidden transitions

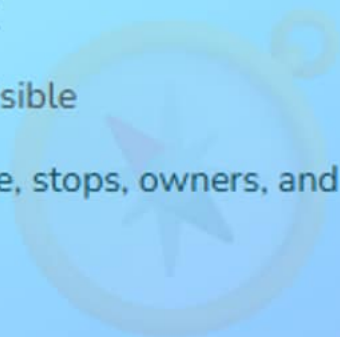
No-op and blocked states remain visible in repository evidence.

PART 1

Lifecycle Contract

Make the state machine visible

Label candidates, evidence, stops, owners, and compatibility gates.

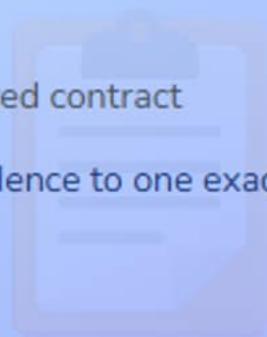


PART 2

Intake

Turn an issue into an approved contract

Follow issue #482 from evidence to one exact approval.

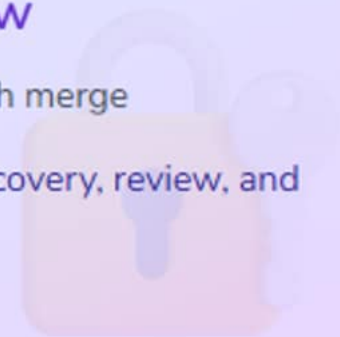


PART 3

Coding and Review

Preserve authority through merge

Expose no-op, blocked recovery, review, and human acceptance.

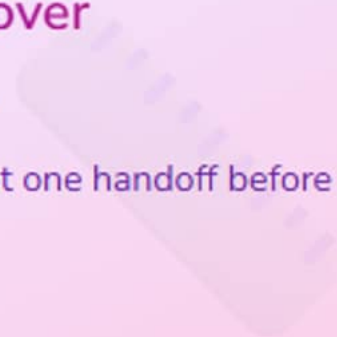


PART 4

Measure and Recover

Measure handoffs locally

Compile, inspect, and pilot one handoff before expansion.



Click any section to jump directly there

Make the State Machine Visible

Define the lifecycle contract while all four workflow sources remain uncompiled candidates.



Candidate Sources

Four sources, not yet runtime proof



Visible State

Evidence, stop, and owner per handoff



Compile Gate

Compatibility before adoption

Platform owner compiles and reviews generated locks
↳ candidate source → compatible workflow

Each Phase Emits Evidence Before It Hands Off

A label exposes state; evidence and named authority permit the transition

Intake	Issue input → intake marker → triaged or visible stop	triage owner
Planning	Intake evidence → fresh plan → planned or visible stop	plan approver
Coding	Exact approval → draft PR → in-review or visible stop	impl owner
Review	Plan + diff + CI → COMMENT → reviewed or visible stop	CODEOWNER

Success labels accumulate; any stop label takes precedence

Labels Record Milestones; They Never Grant Authority Alone

Implied transition

A label appears

The next workflow assumes permission

Missing evidence moves downstream

A failed phase retries invisibly

Evidence-gated handoff

State label and evidence agree

Named actor holds the next decision

Stop reason remains visible

Recovery requires a new workflow run

Four Workflow Sources Are Explicitly Uncompiled Candidates

candidate sources

```
workflows/1-intake.md
workflows/2-planning.md
workflows/3-coding.md
workflows/4-review.md
```

📌 Candidate status

No source is presented as runtime evidence before target-repository compilation.

🔒 Generated locks

The platform owner reviews permissions and commits source plus .lock.yml.

🔑 Schema ownership

The platform owner tracks gh-aw schema changes and recompiles.

Turn an Issue into an Approved Contract

Follow issue #482 from intake evidence to a fresh plan and its named approval boundary.



Intake Evidence

Issue state becomes inspectable



Fresh Plan

Artifact defines bounded work



Exact Approval

Named maintainer grants authority

Issue comment and plan artifact form the handoff
↳ #482 → approved implementation contract

Intake Decides Whether the Issue Is Ready to Plan

Specific, non-duplicative, routable input becomes durable evidence

```
$ issue #482 opened with acceptance criteria
```

```
Intake candidate inspects issue fields and repository paths
```

```
  Candidate duplicates: none found
```

```
  Type: bug
```

```
  Area: payments
```

```
  Routing owner: payments team
```

```
✓ intake result=pass; next owner=issue triage owner
```

```
✓ lifecycle:triaged records the milestone
```

Missing reproduction or ownership → visible needs-input / blocked

Planned Means a Fresh Contract Exists, Not That Coding May Start

plan approval boundary

```
### Approval
Named plan approver: @maintainer
Comment exactly `/approve-plan` to authorize this plan.
```

Bounded scope

Names files, tests, exclusions, expected signals, risk, and rollback.

Fresh evidence

Coding must use the latest plan rather than a superseded artifact.

Named authority

Only @maintainer with repository authority can authorize this plan.

Eight Source Files and Seven Labels Make the Contract Inspectable

uncompiled candidate

```
.github/workflows/  
├── 1-intake.md  
├── 2-planning.md  
├── 3-coding.md  
└── 4-review.md
```

```
instructions/  
├── intake.md  
├── planning.md  
├── coding.md  
└── review.md
```

✓ Milestones

triaged · planned · in-review · reviewed

✗ Stops

needs-input · changes-requested · blocked

⚙ Still candidates

Compilation must generate and validate four matching lock files.

Preserve Authority Through Merge

Contrast authorized progress with visible no-op, blocked recovery, agent comment, and human merge acceptance.



Approval Gate

Only exact authority advances



Visible Stops

No-op and blocked are evidence



Human Merge

CODEOWNER accepts or rejects

Automation proposes and checks; named people decide
↳ COMMENT ≠ approval ≠ merge

The Same Command Has Different Results When Authority Differs

The transition depends on exact text, fresh evidence, and actor authority

✗ Unauthorized request

- 1 Post /approve-plan
Actor lacks repository authority
- 2 Verify contract
Authority check fails
- 3 Emit noop
Reason is observable
- 4 Leave state unchanged
No branch, PR, or partial write

0 repository state changes

✓ @maintainer approval

- 1 Post /approve-plan
Exact command from @maintainer
- 2 Verify contract
planned label + latest plan are fresh
- 3 Start coding
Only approved scope is delegated
- 4 Record approval URL
Draft PR carries provenance

1 bounded implementation authorized

Exact command + fresh plan + named authority

Implementation Authority Ends at One Bounded Draft Pull Request

candidate safe output

```
safe-outputs:  
  create-pull-request:  
    title-prefix: [lifecycle]  
    labels: [agent-generated, lifecycle:in-review]  
    draft: true  
    max: 1
```



Approval evidence

The PR records approver, approval URL, linked issue, and latest plan.



Validation evidence

Exact commands and results travel with files changed and deviations.



No forced progress

Failed checks, stale approval, or scope drift stop implementation.

A Failed Integration Check Stops Visibly Before Review Reruns

✗ Visible stop

Required integration check fails

lifecycle:blocked takes precedence

Stop evidence names the failing signal

💡 Owned recovery

Implementation owner repairs the defect

Human-authored change supplies new evidence

Owner removes the stop label and starts a new run

✓ Recoverable handoff

Review reruns against fresh evidence

No discussion silently restarts work

Resume timestamp enters the recovery ledger

Agent Review Synthesizes Evidence; a Human Owns Merge Acceptance

Agent review submits COMMENT

Compare diff with approved plan

Map acceptance criteria to checks

Report findings and residual risk

Set reviewed only when evidence is ready

CODEOWNER or named reviewer decides

Accept or reject residual risk

Return drift to planning

Return defects to coding

Merge only when repository rules agree

Measure the Handoffs Locally

Define local timing and recovery evidence, then compile and pilot one handoff before expanding.



Local Timing

Repository supplies the measures



Recovery Ledger

Every stop names an owner



Pilot Decision

Inspect one handoff first

First audience action

↳ compile → inspect → pilot

Use Repository Timestamps, Sample Sizes, and Every Stopped Run



Timing definitions

Intake: issue created → intake marker

Planning: intake pass → plan marker

Coding: authorized approval → draft PR

Review: PR sync → review marker



Recovery ledger

Phase, timestamp, and stop reason

Owner able to recover work

Evidence required to resume

Resume timestamp or final disposition

Compilation Is the Compatibility Gate

target repository

gh aw compile

1 Compile four sources

Use the target repository installed gh-aw release.

2 Inspect four locks

Review read-only jobs, safe outputs, permissions, and schema fit.

3 Pilot one handoff

Proceed only when all four candidate sources compile successfully.

● Stop on schema mismatch, undeclared write path, missing artifact, unowned recovery, or merge bypass.

From Implied Automation to Evidence-Gated Handoffs

Before

Workflow sources treated as runnable

Approval inferred from conversational intent

Stops disappear into logs or retries

Automation appears to own merge acceptance

After

Four sources labeled uncompiled candidates

One exact maintainer approval grants coding authority

No-op and blocked recovery remain visible

Agent comments; CODEOWNER retains merge ownership

4

candidate sources compiled together

1

exact approval boundary

0

state changes from unauthorized requests

✓ What You Can Do Today

Today

- Compile all four candidate workflow sources
- Inspect generated lock files and permissions
- Confirm exact approval and merge owners

This Week

- Pilot issue #482 through one handoff
- Record visible no-op and blocked-state evidence
- Assign recovery to a named implementation owner

This Month

- Measure label-to-run latency locally
- Review cost and recovery evidence
- Expand only after the compatibility gate holds

🔑 Key Takeaway

Compilation is the compatibility gate and the first action, not a footnote after adoption.

Official documentation

[GitHub Agentic Workflows overview](#)

Markdown workflow model and core concepts.

[How Agentic Workflows work](#)

Compilation, lock files, execution, and audit markers.

[Security architecture](#)

Read-only agent execution and isolated write handling.

[Safe outputs](#)

Constrained repository output types and validation.

GitHub governance

[Copilot coding agent](#)

Repository tasks return through pull requests for review.

[GitHub Actions workflow syntax](#)

Events, permissions, expressions, and execution semantics.

[About code owners](#)

File ownership and human review routing.



Agentic Lifecycle Orchestration

Evidence-Gated Handoffs from Issue to Pull Request

Compile

Prove four candidate sources are compatible before adoption.

Authorize

Require one exact maintainer approval for implementation.

Keep ownership human

Expose stops and leave merge acceptance with the CODEOWNER.

Which single handoff will you compile, inspect, and pilot first?